Early Detection of Network Attacks on CIC-IDS2017

Personal learning project inspired by SIH 2026 PS 26153 (AI-based network attack forecasting).

1. Summary

I set out to *forecast* attacks (predict an attack within the next 6 minutes). On CIC-IDS2017 at 1-minute resolution this is at chance level, so I reframed the problem as **early detection**: how many minutes after an attack starts can a model flag it, at a fixed false-alarm rate?

Main results (blocked 5-fold CV, 2% benign false-alarm rate, 41 attack episodes):

- Forecasting: no signal before attack onset (negative result, kept in this write-up).
- Detection with 13 features: the best model (logistic regression) catches **34%** of episodes, with a median delay of 1 minute. With the original 8 features the best was about 20%.
- Learned models are only modestly better than a simple anomaly heuristic (27%). The LSTM (22%) does not beat the baselines.

2. Data and pipeline

- Data: CIC-IDS2017 "GeneratedLabelledFlows" CSVs (the versions with timestamps and IPs). 2.83M flows after dropping 288,602 rows with invalid timestamps.
- Flows are aggregated into **60 s windows**, giving 2,453 windows, of which 264 (10.9%) contain attack traffic. Windows are segmented at overnight gaps so no sequence spans a gap.
- Features per window (13, log1p-transformed): traffic-volume features (e.g. packet rate, SYN ratio) plus 5 concentration features: `unique_src_ips`, `top_src_share`, `top_dstport_share`, `top_pair_share`, `mean_log_duration`.
- Samples are the last 6 minutes of windows (lookback = 6), giving 2,428 samples.
- Evaluation: blocked 5-fold cross-validation with a 6-window purge around fold boundaries to stop overlapping sequences from leaking between train and test. Alert threshold is set so that 2% of benign minutes raise a false alarm. An episode counts as detected if any alert fires during it; delay is measured from episode start.
- Models: anomaly heuristic (max |z-score|), logistic regression, XGBoost (TreeSHAP for explanations), LSTM.

3. Negative result: forecasting

Task: predict whether an attack begins within the next 6 minutes. Result: at chance. Looking at the 6 windows before onset for DDoS, DoS Hulk and PortScan, there is no ramp: attacks start abruptly. Contributing factors:

- Only 41 episodes, so any pre-onset signal would be hard to learn.
- Timestamps are minute-resolution, so any sub-minute lead-up is invisible.
- CIC-IDS2017 attacks were launched by script at scheduled times, without a realistic reconnaissance phase.

A synthetic dataset with a planted reconnaissance ramp (`src/make_synthetic_data.py`) is trivially separable. It only demonstrates the method and says nothing about real forecasting ability. **I do not claim forecasting works.**

4. Detection results

model	PR-AUC	ROC-AUC	episodes detected	<=1 min	<=3 min	median delay
anomaly (max abs z)	0.338	0.650	27%	24%	24%	0.0 min
logistic regression	**0.659**	**0.866**	**34%**	20%	24%	1.0 min
XGBoost	0.460	0.787	32%	17%	24%	1.0 min
LSTM	0.624	0.839	22%	12%	17%	1.0 min

Random PR-AUC (base rate) is 0.109.

Observations:

- Concentration features lifted detection from about 20% to 34% of episodes.
- Logistic regression beating XGBoost and the LSTM suggests the limit is data size (41 episodes), not model capacity.
- PR-AUC and episode detection rank the models differently: logistic regression's PR-AUC advantage is large, but its episode-level advantage over the heuristic is modest.

Per attack type (logistic regression)

attack type	episodes	detected	<=1 min	median delay
Bot	10	10%	0%	4.0 min
DDoS	1	100%	0%	4.0 min
DoS GoldenEye	2	100%	100%	0.5 min
DoS Hulk	2	50%	50%	0.0 min
DoS Slowhttptest	5	60%	40%	1.0 min
DoS slowloris	1	100%	100%	0.0 min
FTP-Patator	1	100%	0%	10.0 min
PortScan	7	29%	29%	0.0 min
SSH-Patator	2	50%	0%	4.0 min
Web Attack - Brute Force	7	14%	0%	2.0 min
Web Attack - XSS	3	0%	0%	n/a

High-volume DoS/DDoS attacks are detected quickly. Low-and-slow or application-layer attacks (Bot, web attacks, Patator) are mostly missed: they make up a small share of flows in a window and look like normal traffic in aggregate statistics.

5. Explainability (XGBoost, TreeSHAP)

SHAP values come from XGBoost's built-in TreeSHAP, computed on out-of-fold samples only (2,428 samples). Each sample has 52 inputs: 13 window features, each summarised over the 6-minute lookback as `last`, `mean`, `max` and `slope`. Below, importance is mean |SHAP| summed over the four summaries of each feature and shown as a share of the total. Note that XGBoost is not the best model on PR-AUC (logistic regression is), so these explain XGBoost's behaviour, not necessarily the best detector's.

Which features matter

feature	share of total importance
top_dstport_share	18.6%
flow_count	12.2%
top_pair_share	11.4%
top_src_share	10.3%
mean_log_duration	8.0%
packet_rate	7.2%
src_ip_entropy	7.0%
byte_ratio_asymmetry	6.1%
syn_ratio, unique_dst_ports	5.3% each
unique_src_ips, byte_rate, unique_dst_ips	3.7%, 2.7%, 2.2%

- **The new concentration features dominate:** `top_dstport_share`, `top_pair_share`, `top_src_share` and `mean_log_duration` account for about 48% of importance, and `top_dstport_share` is the single most important feature. This is consistent with the lift in detection from about 20% to 34% after adding them.

- **Raw volume features matter less:** `byte_rate` and `unique_dst_ips` are near the bottom. `flow_count` is the exception.
- **Window summary matters:** the 6-minute `mean` carries about 45% of importance, `max` 25%, `last` 20% and `slope` about 10%. The model relies mostly on sustained level shifts rather than on trends, which fits the finding that attacks start abruptly with no ramp.

What drives detection per attack type (mean signed SHAP over attack-window samples, features pushing the score up)

attack type	main features raising the score	notes
DDoS, DoS Hulk, DoS GoldenEye	`mean_log_duration`, `top_pair_share`, `flow_count`	`mean_log_duration` is by far the largest contributor (about +2.2 to +2.7). Short flows to one src/dst pair.
DoS Slowhttptest, slowloris	`top_pair_share`, `flow_count`, `mean_log_duration`	Same features but much smaller contributions, so these are caught with less margin.
Bot, XSS	`top_dstport_share`	`flow_count` pushes the score *down* (-1.06 for Bot, -1.33 for XSS): these windows look quieter than normal, which is why they are missed.
PortScan	`top_dstport_share`, `flow_count`, `src_ip_entropy`	`mean_log_duration` pushes the score down (-0.99).
FTP/SSH-Patator, Web Brute Force	`top_src_share`, `syn_ratio`, `flow_count`	All contributions are small (mostly under 0.35).

Caveats: these are sample-level averages (one sample per minute, so long episodes weigh more, e.g. 57 FTP-Patator samples), and SHAP describes how the model uses features, not causal properties of the attacks. The pattern for Bot and the web attacks is the useful one: the features that would detect them either contribute very little or point the wrong way, which supports the future-work item on per-flow and payload-level features. The dashboard shows per-alert SHAP explanations for the XGBoost model.

6. Limitations

- **Very small sample:** 41 episodes. Several attack types have 1-2 episodes, so per-type percentages are anecdotal and confidence intervals are wide.
- **Single dataset, one week**, with scripted attacks and a lab topology. Results may not transfer to real networks.
- **Episode-level metric:** an episode is "detected" if any alert fires in it, which is lenient. Per-minute recall is much lower (the dashboard for XGBoost shows 76 attack minutes flagged vs 188 missed).
- Threshold chosen on out-of-fold scores at a fixed false-alarm rate; no separate held-out test week.
- No hyperparameter search was done for the LSTM, so its result is a floor, not a ceiling.

7. Dashboard

`streamlit run dashboard/app.py` shows out-of-fold risk scores over time with the alert threshold, actual attack minutes, flagged/missed/false-alarm counts, and a per-attack-type table, with a model selector.

8. Future work

- Per-flow or sub-minute features to catch Bot and web attacks (payload and URL-level features are not in these flow statistics).
- Per-source-IP profiles rather than whole-network windows.
- A longer or more varied dataset to give sequence models enough episodes, and a time-based held-out test set.
- Revisit forecasting only with data that contains realistic reconnaissance phases.